QRAEMON

Embedded Product

From Prototype Computer to a Production Onboard Artificial Brain

Product definition

QRAEMON Embedded is the onboard physical embodiment of the QRAEMON cognitive runtime: dedicated compute, persistent storage, robot interfaces, security, power, thermal management, and software packaged into a deployable module that can operate locally inside a robot without depending on an external real-time computer or cloud.

Engineering Development Blueprint

Software-Proven Brain → Compute Prototype → Embedded Prototype → Engineering Validation → Pilot Hardware → Production Module

Version 1.0
QRAEMON Internal Technical Strategy

Contents

1 Executive Summary **3**

2 Product Definition **3**

3 Core Hardware Principle **4**

4 Stage 0 — Freeze the Software Requirements **4**

5 Stage 1 — Define the Embedded Workload **4**

6 Stage 2 — Establish the Compute Budget **5**

7 Stage 3 — Select Reference Compute **5**

8 Stage 4 — Build Embedded Prototype V0 **6**

9 Stage 5 — Define the Hardware Architecture **7**

10 Stage 6 — Compute Subsystem **7**

11 Stage 7 — Persistent Memory Hardware **7**

12 Stage 8 — Memory Storage Architecture **8**

13 Stage 9 — Offline-First Operation **8**

14 Stage 10 — Robot I/O Architecture **9**

15 Stage 11 — Time and Synchronization **9**

16 Stage 12 — Thermal Engineering **10**

17 Stage 13 — Power Architecture **10**

18 Stage 14 — Safe Shutdown **11**

19 Stage 15 — Security Architecture **11**

20 Stage 16 — Mechanical Integration **11**

21 Stage 17 — EMI / EMC Engineering **12**

22 Stage 18 — Embedded Software Stack **12**

23 Stage 19 — Real-Time Boundary **13**

24 Stage 20 — Hardware-in-the-Loop Testing **13**

25 Stage 21 — Reliability Qualification **13**

26 Stage 22 — Prototype Generations **14**

26.1 V0 — Development system 14

26.2 V1 — Engineering prototype 14

26.3 V2 — Pilot hardware 14

26.4 V3 — Production module 15

27 Stage 23 — Hardware Validation Matrix 15
28 Stage 24 — Prototype Robot Integration 15
29 Stage 25 — Benchmark Embedded vs External Computer 16
30 Stage 26 — Edge Deployment 16
31 Stage 27 — Fleet Experience 16
32 Stage 28 — Embedded Update System 17
33 Stage 29 — Manufacturing Strategy 17
34 Stage 30 — Supply Chain Strategy 18
35 Stage 31 — Product Variants 18
36 Stage 32 — Hardware–Software Co-Design 18
37 Stage 33 — Certification and Compliance 19
38 Stage 34 — Production Test System 19
39 Stage 35 — Device Identity 20
40 Stage 36 — Field Diagnostics 20
41 Stage 37 — Hardware Roadmap 21
42 Stage 38 — Team Required 21
43 Stage 39 — Economic Strategy 22
44 Stage 40 — What Must Not Be Built Early 22
45 Stage 41 — Critical Engineering Risks 23
46 Stage 42 — The Long-Term Hardware Vision 23
47 Stage 43 — Final Development Gates 24
48 Definition of Embedded Success 24
49 Final QRAEMON Product Strategy 25

1 Executive Summary

QRAEMON Embedded should not be the first thing QRAEMON builds.

The correct sequence is:

First prove the cognitive architecture in software. Then engineer the hardware around the measured requirements.

The embedded product exists to solve a different problem from the software product.

QRAEMON Software asks:

Can the artificial brain work?

QRAEMON Embedded asks:

Can that brain become a reliable physical subsystem of a machine?

The hardware therefore should not be designed around theoretical performance.

It should be designed around measured workload.

2 Product Definition

QRAEMON Embedded consists of:

- compute;
- persistent storage;
- memory subsystem;
- robot I/O;
- networking;
- security;
- power management;
- thermal management;
- physical enclosure;
- onboard QRAEMON runtime.

It should provide the robot with a persistent local cognitive layer.

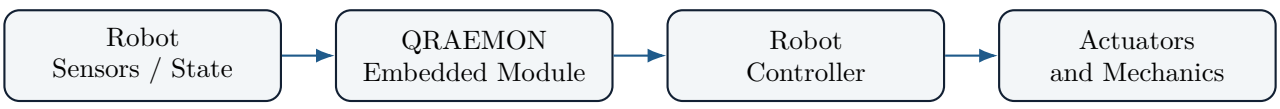


Figure 1: QRAEMON Embedded inside the robot architecture.

The fast physical control loop remains with the robot controller.

3 Core Hardware Principle

The most important hardware principle is:

Do not design custom hardware before understanding the actual computational workload.

The development order should therefore be:

$$\text{Software} \rightarrow \text{Profiling} \rightarrow \text{Reference Hardware} \rightarrow \text{Prototype} \rightarrow \text{Custom Hardware}$$

not:

$$\text{Idea} \rightarrow \text{Custom PCB}$$

4 Stage 0 — Freeze the Software Requirements

Before hardware selection, measure:

- CPU utilization;
- GPU/NPU utilization;
- RAM usage;
- accelerator memory;
- storage bandwidth;
- storage growth;
- inference latency;
- memory retrieval latency;
- planner latency;
- perception latency;
- communication latency;
- power consumption;
- thermal load.

The result should be a workload specification.

5 Stage 1 — Define the Embedded Workload

The workload should be separated into computational classes.

Workload	Requirement
Perception	Accelerator-friendly inference
Memory retrieval	Low-latency storage/index access
Reasoning	General compute / accelerator depending on model
Planning	Deterministic bounded execution
Experience recording	Persistent storage throughput
Consolidation	Background compute
Logging	Reliable sequential storage
Robot communication	Predictable I/O
Security	Cryptographic operations and secure identity

This prevents selecting hardware based on a single benchmark.

6 Stage 2 — Establish the Compute Budget

The compute budget should be derived experimentally.

Define:

$$C_{\text{required}} = C_{\text{perception}} + C_{\text{memory}} + C_{\text{reasoning}} + C_{\text{planning}} + C_{\text{system}}$$

and include safety margin:

$$C_{\text{design}} = MC_{\text{required}}$$

where $M > 1$ is the engineering margin.

The same principle applies to:

- RAM;
- storage;
- power;
- thermal capacity;
- communication bandwidth.

7 Stage 3 — Select Reference Compute

The first embedded prototype should use an existing industrial or developer-grade compute module.

Selection criteria:

- AI acceleration;
- CPU performance;
- RAM capacity;
- persistent storage interface;

- camera interfaces;
- Ethernet;
- USB;
- CAN or required fieldbus support;
- operating-system support;
- long-term availability;
- thermal characteristics;
- power envelope;
- development ecosystem.

Do not optimize for unit cost yet.

Optimize for:

$$\text{availability} + \text{development speed} + \text{measurability} + \text{reliability}.$$

8 Stage 4 — Build Embedded Prototype V0

V0 should be physically ugly but technically useful.

It may consist of:

- reference compute board;
- external SSD;
- standard power supply;
- development carrier;
- Ethernet;
- USB;
- sensor connections;
- simple cooling;
- temporary enclosure.

The objective is not industrial design.

The objective is to prove:

> **The complete QRAEMON cognitive runtime can operate locally inside a robot using only onboard resources.**

9 Stage 5 — Define the Hardware Architecture

The hardware should eventually contain the following logical blocks.

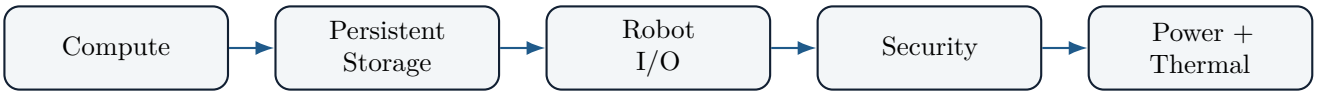


Figure 2: Logical QRAEMON Embedded hardware blocks.

This diagram is conceptual rather than a literal PCB topology.

10 Stage 6 — Compute Subsystem

The compute subsystem must support:

- perception;
- memory retrieval;
- reasoning;
- planning;
- experience evaluation;
- background consolidation;
- security;
- observability.

The design question is not:

"What is the fastest processor?"

It is:

"What compute architecture provides sufficient cognition per watt?"

That distinction becomes increasingly important for mobile robots.

11 Stage 7 — Persistent Memory Hardware

QRAEMON's persistent memory is not merely RAM.

The embedded system needs storage that survives:

- shutdown;
- power interruption;
- reboot;
- software restart;

- controlled update.

The storage architecture should support:

- transactional writes;
- journaling;
- integrity checking;
- versioning;
- recovery;
- wear management;
- bounded storage growth.

12 Stage 8 — Memory Storage Architecture

The physical storage hierarchy should separate workloads.

A conceptual arrangement is:

Layer	Purpose
RAM	Active cognitive state
Fast local storage	Indexes and frequently accessed memory
Persistent storage	Long-term experiences
External/cloud backup	Optional fleet-level synchronization

The cloud must not be a prerequisite for local cognition.

13 Stage 9 — Offline-First Operation

The embedded product should be able to operate without Internet access.

The core loop should be:



Figure 3: Local cognitive operation without cloud dependence.

Cloud services may later support:

- fleet analytics;
- backups;
- software updates;
- research;
- aggregated learning;

- remote diagnostics.

But loss of network connectivity should not automatically destroy the robot's cognitive capability.

14 Stage 10 — Robot I/O Architecture

The embedded device should communicate through an abstraction layer.

Potential interfaces include:

- Ethernet;
- USB;
- CAN;
- fieldbus;
- serial;
- GPIO;
- camera interfaces;
- time synchronization interfaces.

The architecture should be:

$$\text{Physical Interface} \rightarrow \text{Driver} \rightarrow \text{Robot Adapter} \rightarrow \text{Canonical QRAEMON Interface}$$

This prevents hardware-specific protocols from entering the cognitive architecture.

15 Stage 11 — Time and Synchronization

Experience is fundamentally temporal.

Therefore the embedded system must maintain reliable timestamps.

At minimum, synchronize:

- camera data;
- robot state;
- action execution;
- action completion;
- failure events;
- memory writes;
- safety events.

Without temporal consistency, experience reconstruction becomes unreliable.

16 Stage 12 — Thermal Engineering

AI compute generates heat.

The mechanical design therefore becomes part of the cognitive architecture.

The thermal chain is:

$$\text{Compute} \rightarrow \text{Heat Spreader} \rightarrow \text{Cooling System} \rightarrow \text{Enclosure} \rightarrow \text{Environment}$$

Design for:

- continuous operation;
- peak inference;
- sustained consolidation;
- ambient temperature;
- dust;
- vibration;
- serviceability.

Thermal throttling must not silently destroy real-time cognitive performance.

17 Stage 13 — Power Architecture

The embedded system must define:

- input voltage range;
- peak power;
- average power;
- startup current;
- shutdown behavior;
- brownout behavior;
- battery interaction;
- power sequencing.

The design should explicitly separate:

$$P_{\text{average}}$$

from

$$P_{\text{peak}}.$$

Peak compute events can otherwise cause system instability.

18 Stage 14 — Safe Shutdown

Persistent memory creates a new engineering requirement.

The system must survive unexpected shutdowns.

A safe transaction should conceptually follow:

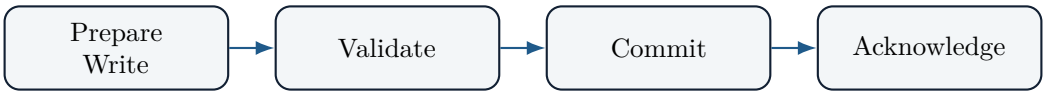


Figure 4: Conceptual persistent-memory transaction.

Incomplete writes must be recoverable.

19 Stage 15 — Security Architecture

An onboard artificial brain is a high-value computational component.

Security should include:

- secure boot;
- signed firmware;
- signed software;
- signed model packages;
- device identity;
- secure key storage;
- encrypted persistent memory;
- authenticated updates;
- rollback protection;
- local authorization;
- tamper-evident logs.

The security principle should be:

No untrusted software should silently become part of the brain.

20 Stage 16 — Mechanical Integration

Because the product is physically installed in robots, mechanical engineering matters.

The enclosure should eventually specify:

- mounting points;
- connector locations;

- service access;
- cooling path;
- cable routing;
- vibration isolation;
- ingress protection requirements;
- EMI considerations;
- mass;
- center of gravity.

For mobile robots, mass and volume become increasingly important.

21 Stage 17 — EMI / EMC Engineering

As compute density increases, electrical noise becomes relevant.

The production design should evaluate:

- electromagnetic emissions;
- susceptibility;
- high-speed interfaces;
- power-converter noise;
- cable shielding;
- grounding;
- connector integrity.

This should be addressed before final certification.

22 Stage 18 — Embedded Software Stack

The embedded software stack should be layered.

Layer	Responsibility
Application	Robot task configuration
QRAEMON Runtime	Cognitive processing
Memory Engine	Persistent experience
Adapter Layer	Robot integration
Middleware	Inter-process communication
Operating System	Resource management
Drivers	Hardware interfaces
Boot / Security	Platform trust
Hardware	Compute and I/O

The QRAEMON cognitive architecture should remain independent of the specific operating system wherever practical.

23 Stage 19 — Real-Time Boundary

QRAEMON Embedded must not accidentally become the low-level servo controller.

The boundary should remain:

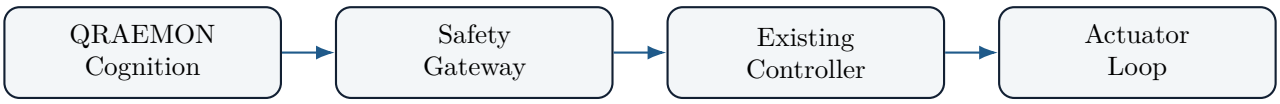


Figure 5: Cognitive-to-control boundary.

The existing deterministic controller remains responsible for high-frequency control.

This dramatically reduces the initial safety and certification problem.

24 Stage 20 — Hardware-in-the-Loop Testing

Before installing the module into a valuable robot, use hardware-in-the-loop testing.

Test:

- sensor loss;
- controller loss;
- communication delay;
- corrupted packets;
- storage failure;
- power interruption;
- thermal throttling;
- processor overload;
- memory corruption;
- invalid actions.

The system must fail conservatively.

25 Stage 21 — Reliability Qualification

The embedded product needs tests beyond software correctness.

Test:

- continuous runtime;
- repeated power cycles;

- storage endurance;
- thermal cycling;
- vibration;
- connector insertion/removal;
- boot reliability;
- software rollback;
- update recovery;
- communication recovery.

The objective is to discover failures before customers do.

26 Stage 22 — Prototype Generations

The hardware should evolve through several generations.

26.1 V0 — Development system

- reference compute;
- development storage;
- external power;
- open enclosure.

Purpose: prove local cognition.

26.2 V1 — Engineering prototype

- custom carrier board where useful;
- integrated power;
- reliable storage;
- controlled cooling;
- robot connectors;
- preliminary enclosure.

Purpose: prove robot integration.

26.3 V2 — Pilot hardware

- refined PCB;
- production-oriented connectors;
- improved thermal path;
- security hardware;

- serviceable enclosure;
- manufacturing documentation.

Purpose: external deployment.

26.4 V3 — Production module

- optimized BOM;
- manufacturable PCB;
- validated supply chain;
- environmental qualification;
- production test fixtures;
- certification as required.

Purpose: commercial scale.

27 Stage 23 — Hardware Validation Matrix

Test category	Pass condition
Compute	Required workload sustained
Latency	Cognitive cycle within target
Memory	Persistent writes remain reliable
Storage	Required lifetime and throughput
Thermal	No unacceptable throttling
Power	Stable across operating conditions
I/O	Reliable robot communication
Security	Unauthorized software rejected
Recovery	Safe reboot and rollback
Mechanical	Survives intended environment

28 Stage 24 — Prototype Robot Integration

The first embedded robot should be the same controlled platform used to validate QRAEMON Software.

This is strategically important.

It allows:

$$\text{Software Architecture} \rightarrow \text{Embedded Architecture}$$

without introducing an entirely new robotics problem.

The embedded device should reproduce the same cognitive behavior that already passed software validation.

29 Stage 25 — Benchmark Embedded vs External Computer

The embedded system should be compared against the software running on a conventional external computer.

Measure:

- task success;
- cognitive latency;
- memory latency;
- power consumption;
- thermal stability;
- startup time;
- recovery time;
- network independence;
- persistent-memory reliability.

The objective is not necessarily to outperform a large external computer.

The objective is to prove that the onboard product is sufficiently capable and substantially more deployable.

30 Stage 26 — Edge Deployment

After laboratory validation, install QRAEMON Embedded in a real operational environment.

The deployment should test:

- dust;
- vibration;
- temperature;
- intermittent networking;
- unexpected restarts;
- long operating periods;
- repeated task cycles;
- memory growth.

This is where laboratory assumptions meet physical reality.

31 Stage 27 — Fleet Experience

Once multiple QRAEMON systems exist, a second level of architecture becomes possible.

Each robot has:

Local Experience

while the fleet can optionally exchange:

Validated Shared Experience

The distinction is important.

Not every experience should automatically propagate to every robot.

A future fleet architecture should therefore support:

- experience provenance;
- confidence;
- validation;
- compatibility;
- robot capability matching;
- controlled synchronization.

32 Stage 28 — Embedded Update System

Because QRAEMON is continually evolving, updates are unavoidable.

The update system should support:

1. package verification;
2. compatibility verification;
3. staged installation;
4. backup;
5. activation;
6. health check;
7. automatic rollback.

A failed update must not brick the robot.

33 Stage 29 — Manufacturing Strategy

Do not establish a large manufacturing operation immediately.

Use:

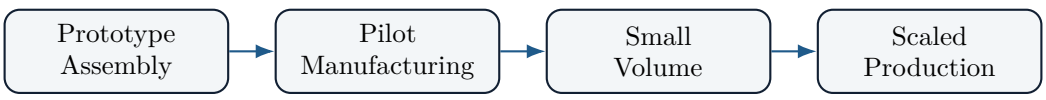


Figure 6: Hardware manufacturing progression.

Every step should be justified by actual demand.

34 Stage 30 — Supply Chain Strategy

Identify single points of failure.

Track:

- compute module availability;
- memory components;
- storage;
- connectors;
- power electronics;
- cooling components;
- security components;
- PCB fabrication;
- enclosure manufacturing.

For critical components, establish second-source strategies where economically justified.

35 Stage 31 — Product Variants

Do not create many variants too early.

The eventual product family may become:

Variant	Target
QRAEMON Edge	Low-power robots
QRAEMON Standard	Industrial manipulators
QRAEMON Performance	High-compute mobile robots
QRAEMON Rugged	Harsh environments
QRAEMON Custom	Strategic OEM integration

However, these are future product possibilities, not initial requirements.

36 Stage 32 — Hardware–Software Co-Design

Once deployments become large enough, the software and hardware should be optimized together.

The optimization target becomes:

$$\text{Intelligence per Watt}$$

rather than merely:

Compute per Watt.

Possible future research directions include:

- specialized memory acceleration;
- efficient vector retrieval;
- dedicated perception acceleration;
- heterogeneous compute;
- low-power persistent storage;
- neuromorphic experiments;
- FPGA acceleration;
- custom silicon only when justified.

Custom silicon should be the final consequence of a proven workload, not the starting point.

37 Stage 33 — Certification and Compliance

The exact requirements depend on target geography, robot type, installation environment, voltage, communications, and application.

Therefore the company should create a compliance matrix covering:

- electrical safety;
- electromagnetic compatibility;
- radio requirements where applicable;
- environmental requirements;
- machinery/robot safety;
- cybersecurity;
- functional safety interfaces;
- documentation.

Do not design certification after the final PCB has already been frozen.

38 Stage 34 — Production Test System

Every manufactured unit should pass automated tests.

Test:

- boot;
- CPU;

- accelerator;
- RAM;
- storage;
- Ethernet;
- robot I/O;
- temperature sensors;
- power rails;
- security identity;
- software integrity.

A production device should leave the factory with a known-good state.

39 Stage 35 — Device Identity

Every embedded QRAEMON should have a unique identity.

That identity should associate:

- hardware serial;
- security credentials;
- software version;
- memory version;
- configuration;
- deployment history;
- maintenance history.

This becomes essential when hundreds or thousands of robots are deployed.

40 Stage 36 — Field Diagnostics

A production embedded brain must be diagnosable.

Collect:

- CPU load;
- accelerator load;
- memory usage;
- storage health;
- temperature;
- power state;

- software health;
- cognitive latency;
- memory retrieval statistics;
- action failures.

Diagnostics should not expose unnecessary private or sensitive operational information.

41 Stage 37 — Hardware Roadmap

The complete progression is:

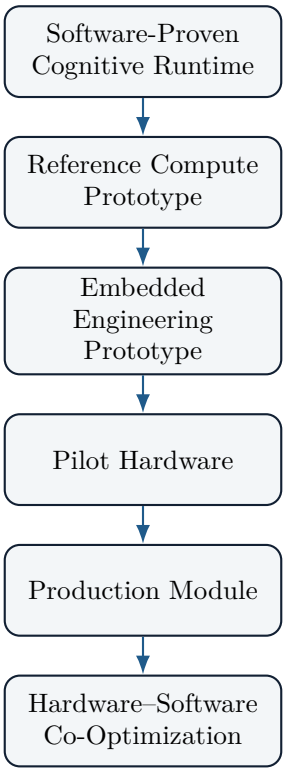


Figure 7: QRAEMON Embedded product evolution.

42 Stage 38 — Team Required

The hardware team should arrive after the software workload becomes measurable.

Role	Responsibility
Embedded Systems Engineer	Platform and operating system
Computer Architecture Engineer	Compute and acceleration
Electronics Engineer	Power, PCB and interfaces
Firmware Engineer	Boot, drivers and hardware control
Mechanical Engineer	Enclosure, cooling and mounting
Thermal Engineer	Heat transfer and reliability
Manufacturing Engineer	Production and test
Security Engineer	Hardware/software trust
Systems Engineer	Overall integration

Initially, several of these responsibilities can be performed by a small interdisciplinary team.

43 Stage 39 — Economic Strategy

The first hardware generation should optimize:

$$\text{Learning Rate}$$

not:

$$\text{Gross Margin}.$$

Early units will be expensive.

That is acceptable if they generate:

- deployment knowledge;
- failure data;
- thermal data;
- workload data;
- integration knowledge;
- customer feedback.

Later generations should optimize:

$$\text{Cost} + \text{Power} + \text{Size} + \text{Reliability}.$$

44 Stage 40 — What Must Not Be Built Early

Do not initially build:

- a custom AI chip;

- a custom processor;
- a highly specialized PCB before workload measurement;
- a massive storage subsystem;
- a universal robot connector;
- an expensive industrial enclosure;
- multiple hardware variants;
- a complete robot;
- a custom humanoid platform.

The first embedded product should be a compact, reliable computer that proves QRAEMON can become a physically deployable brain.

45 Stage 41 — Critical Engineering Risks

Risk	Mitigation
Insufficient compute	Profile before hardware freeze
Excessive power	Optimize workload and accelerators
Thermal throttling	Thermal modeling and sustained-load tests
Storage failure	Transactional persistence and redundancy
Network dependence	Offline-first architecture
Security compromise	Secure boot and signed software
Hardware obsolescence	Select long-life platforms
Supply-chain shortage	Qualify alternative components
Mechanical failure	Environmental testing
Overengineering	Gate hardware complexity by evidence

46 Stage 42 — The Long-Term Hardware Vision

The long-term objective is not to sell a computer box.

The computer is only the physical substrate.

The deeper architecture is:

$$\text{Cognition} + \text{Persistent Memory} + \text{Compute} + \text{Interfaces} + \text{Embodiment}$$

Eventually, the same QRAEMON cognitive architecture should be able to operate on different physical platforms.

That requires:

Stable cognition + replaceable embodiment

The hardware should therefore become progressively more optimized while the cognitive architecture remains conceptually continuous.

47 Stage 43 — Final Development Gates

Gate	Required evidence
H0 Workload	Software requirements measured
H1 Reference Compute	Local cognition works
H2 V0	Embedded operation demonstrated
H3 Engineering Prototype	Power, thermal and I/O validated
H4 HIL	Hardware failures handled safely
H5 Pilot Hardware	Long-duration robot deployment
H6 Production Design	Manufacturable and testable
H7 Qualification	Environmental/reliability validation
H8 Production	Repeatable manufacturing
H9 Fleet	Multiple devices operate reliably

48 Definition of Embedded Success

QRAEMON Embedded succeeds when:

1. the full QRAEMON cognitive runtime operates onboard;
2. the robot does not require cloud access for its core cognitive loop;
3. persistent experience survives power cycles;
4. the module integrates with multiple robot architectures;
5. the hardware operates continuously within thermal limits;
6. software updates are secure and recoverable;
7. hardware failures produce controlled behavior;
8. the same cognitive architecture can move between embodiments;
9. production units can be manufactured and tested reliably.

The embedded product is not "an AI computer."

It is the physical substrate that allows the QRAEMON artificial brain to live inside machines.

49 Final QRAEMON Product Strategy

The two products should ultimately converge into one architecture.

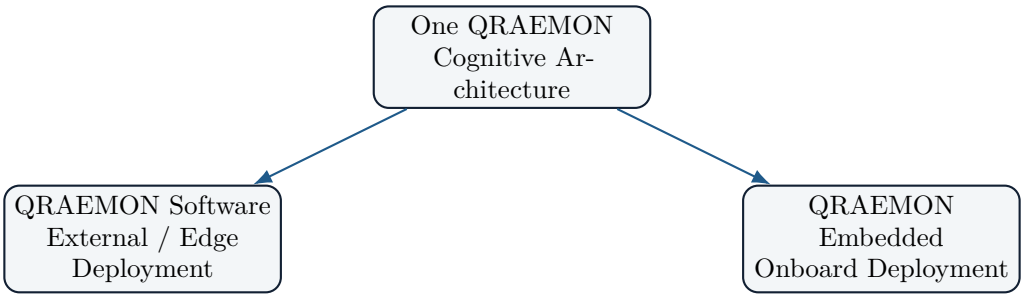


Figure 8: One artificial brain, two deployment forms.

The strategic sequence is therefore:

Prove the brain in software.

Deploy the brain in hardware.

Accumulate experience through both.

Improve the same brain continuously.

The ultimate moat is not the enclosure or the PCB.

It is the combination of:

- cognitive architecture;
- memory architecture;
- experience accumulated through deployment;
- outcome evaluation;
- robot integration knowledge;
- continual-learning mechanisms;
- deployment infrastructure;
- hardware/software co-design.

That creates the intended compounding loop:

$$\boxed{\text{Deploy} \rightarrow \text{Observe} \rightarrow \text{Accumulate} \rightarrow \text{Improve} \rightarrow \text{Redeploy}}$$

This is the technical foundation on which the larger QRAEMON Artificial Brain program can eventually be built.